Sorted Pagination of Authorized Rows in Hierarchical Role-Based Access Control

Ross Slaney

***Abstract*—Business applications often control access through a hierarchy of the resources they manage. In a retail application, products belong to stores, stores to regions and regions to chains. A role is a named set of permissions, such as permission to view products, and assigning a role to a user at a resource makes its permissions apply to that resource and to every resource below it. The same manager role, assigned at a region, lets a regional manager see every product in the region's stores, and, assigned at a store, lets a store manager see only that store's products. SHRBAC enforces this model inside the application's relational database, with an access check that the application adds to its own SQL queries. Listing a manager's products sorted by price, one page at a time, is then expensive when the manager's products are few or sort late. The database reads products in price order and checks each one until the page is full, so it also reads every earlier product from other stores. In a table of 1,000,000 products, the first page of 20 reads 184,205 products on PostgreSQL 16 for a store manager with 100 products, and 980,020 for a chain manager whose 20,000 products have the highest prices. We add, for every index of a table, an ancestor-partitioned projection that lists each row under each of its ancestors in the resource hierarchy, sorted by the index key. A store's products then appear together and in price order, so a page reads the first entries under each resource where the user holds the role, merges them, and verifies the selected rows with the original access check. We prove that the method returns only rows the user may see and misses none while the projections are current, and that the cost of a page depends on the page size, the depth of the hierarchy and the user's role assignments, but not on the size of the table. On SQL Server 2022 and PostgreSQL 16 respectively, the first page for each of the two managers reads 100 and 80 rows. Users with thousands of separate role assignments remain the expensive case, and each projection stores one entry per row and ancestor.**

I. Introduction

Business applications often control access through a hierarchy of the resources they manage. In a retail application, each product belongs to a store, each store to a region and each region to a chain. Access is expressed with roles. A role is a named set of permissions, such as permission to view products or to change prices, and an administrator assigns a role to a user at a particular resource in the hierarchy. The role's permissions then apply to that resource and to every resource below it, which together form the scope of the assignment. A manager role assigned to one user at a region lets that user view every product in every store of the region, and the same role assigned to another user at a single store lets that user view only that store's products. Both users hold the same permissions, and their assignments differ only in the part of the hierarchy they cover.

Azure role-based access control (Azure RBAC) is built on the same relationship between principals, roles, permissions and scopes [1]. A role assignment in Azure consists of a security principal, which is a user, group, service principal or managed identity; a role definition, which is a collection of permissions; and a scope, which is a management group, a subscription, a resource group or a single resource [1], [2]. Scopes form a parent–child hierarchy in which lower levels inherit role permissions from higher levels [3], so a role assigned at a resource group applies to every resource in that group, and a principal's effective permissions are the sum of its role assignments [1]. Classic role-based access control assigns permissions to roles and roles to users [4], [5]; the scope, a resource whose descendants inherit the assignment, is what makes the model hierarchical.

Applications that manage their own data use the same model over resources that they define themselves, such as the chains, stores and products of a retailer or the workspaces, projects and documents of a collaboration tool. Authorization services for developers provide this inheritance. WorkOS arranges an application's resources in a hierarchy down which permissions flow [6], OpenFGA lets the editors of a folder edit the documents it contains [7], and Zanzibar expresses inheritance from parent objects through userset rewrite rules [8].

SHRBAC [9] enforces this model inside the application's relational database. The resource hierarchy, the roles and the role assignments, which SHRBAC calls grants, are tables next to the application's data, and each protected row, such as a product, records the resource it belongs to. An access check decides whether a user may access a row by walking from the row's resource up the hierarchy and looking, at each resource on the way, for a grant to the user or to one of the user's groups whose role carries the requested permission. The check is a SQL function that the application adds to its own queries, so a single SQL statement can filter, sort and page the rows that a user may see. The concepts are those of scoped hierarchical role-based access control; the earlier paper [9] contributes their enforcement inside the database over resources that the application defines, with proofs that the check is sound and complete and a bound on its cost. One check visits at most D ancestors for each of the user's M principals, meaning the user and the user's groups, and on a hierarchy of depth 9 with 1.5 million resources a single check took under 1.5 ms (median).

Checking one row is fast, but listing rows is a different task. Consider the store manager's product list, sorted by price and shown 20 products at a time. The products that qualify are the 100 products of the manager's store. The table, however, holds 1,000,000 products, and the price index that sorts them interleaves the store's products with those of the other 9,999

stores. To produce the first page, the database reads products in price order through the index and applies the access check to each one until 20 pass. Every product of another store that is cheaper than the store's 20th cheapest product is read and rejected, and on PostgreSQL 16 the first page reads 184,205 products (Section VII). The number of rows read depends on where the user's rows fall in the order. For a user who may see a fraction σ of the rows, spread evenly, a page of P rows reads about P/σ rows. For a user whose V rows come last in the order, the first page reads $N - V + P$ rows, which is 980,020 for a chain manager whose 20,000 products have the highest prices. The cost analysis of [9] states this dependence, and its evaluation measured users whose rows are dense in the order. Authorization services limit the same operation. OpenFGA's ListObjects returns at most 1,000 results within 3 seconds by default [10], and SpiceDB's documentation recommends its LookupResources call when a user can access fewer than about 10,000 resources [11].

The products a manager may see are exactly the products below the resources where the manager holds grants, such as the 100 products of a store or the 2,000 of a region. The price index cannot find them without reading other products, because it orders all products together. We add, for every index of a protected table, an ancestor-partitioned index projection, a table that lists each row once under each active ancestor of its resource and sorts the entries under each ancestor by the index key. In the price projection, the store's 100 products appear together under the store, in price order, so the store manager's first page is the first 20 entries under the store. A user with grants on several resources reads the first entries under each of them and merges these sorted lists, as in the merge step of merge sort. Each selected row is then checked again with the original access check, so the projections determine which rows are found but never which rows are allowed. They are updated in the same transaction as every write and can be rebuilt from the application's tables.

This paper makes four contributions.

- An analysis of sorted pagination under hierarchical access control. Checking rows in sort order reads every row up to the P-th one the user may see, and reading the user's rows through an ancestor closure, a table that pairs every row with every ancestor of its resource, reads all of them; for some users, each grows with the size of the table (Section III).
- Ancestor-partitioned index projections and a page algorithm in three steps, a short scan in sort order, a merge of the user's ranges and a final access check, with the resources where the user holds grants, reduced to the highest ones, computed inside the database (Section IV).
- Proofs that the algorithm returns only rows the user may see, whatever the projections contain, and misses none while they are current, and that a page costs $O(P \cdot D \cdot M \cdot G_{\max} + G \cdot D + k \cdot P \log k)$ operations whatever the size of the table, with bounds on storage and write cost (Section V).
- An implementation for SQL Server and PostgreSQL that derives projections from the application's indexes and maintains them in the writing transaction (Section VI), and an evaluation with 1,000,000 rows that compares it with both alternatives for dense, sparse and late-sorting users and for users with many separate grants (Section VII).

Section VIII reviews related work, and Section IX discusses limitations.

II. Background

A. Scoped Hierarchical RBAC in Azure and SHRBAC

SHRBAC uses the access model of Azure RBAC described in Section I and enforces it in a different place (Table I). Azure Resource Manager evaluates each request [1], whereas SHRBAC evaluates a predicate inside the application's queries. The two also differ in the hierarchy, in group semantics and in several features that Azure adds.

Management groups, the highest level of Azure scope, form a tree of up to six levels below the root, and each management group and subscription can have only one parent [12]. A role definition lists the operations that a role permits, its Actions and DataActions, and the operations excluded from them, its NotActions and NotDataActions; an exclusion is not a denial, because another assignment may grant the excluded operation [13]. Assignments to groups are transitive, so a user in a group that belongs to an assigned group receives the assignment [1]. Azure also has features that SHRBAC omits or simplifies. Deny assignments block actions even when a role grants them, and in generally available scenarios Azure creates them rather than customers [14]. Role assignments may carry attribute conditions [2], and Privileged Identity Management assigns roles with start and end dates [15]. To decide a request, Azure Resource Manager collects the role and deny assignments that apply to the resource and blocks the request if a deny assignment applies. Otherwise it determines the roles that the caller and its groups hold, checks whether one of them permits the action, and evaluates any conditions [1].

Earlier models scope role assignments in the same way. ROBAC [16] and RBAC with resource hierarchies [17] attach assignments to the nodes of an organization or resource hierarchy, and [9] relates SHRBAC to these models and to RBAC96 and the NIST RBAC standard.

B. The SHRBAC Model

In SHRBAC, a user may access a row if a role that carries the permission is granted to the user, or to one of the user's groups, at the row's resource or at an active ancestor of it. We restate the definitions of [9] with two conditions that its implementation applies. A resource can be deactivated, for example when a store closes, and an inactive resource stops inheritance, so a grant on the region no longer reaches the closed store's products. A permission can also be restricted to one resource type, so that a permission to change prices applies to products but not to the stores above them.

Resources form a rooted tree of depth at most D, and each resource may be marked inactive. For a resource r, $\mathrm{anc}(r)$

TABLE I
AZURE RBAC AND SHRBAC.

Concept	Azure RBAC	SHRBAC
Principals	users, groups, service principals, managed identities	users, groups, service accounts, agents
Role	role definition: Actions and DataActions, less NotActions and NotDataActions	set of permissions
Scope hierarchy	management groups (up to six levels below the root), subscriptions, resource groups, resources; one parent per node	application-defined tree of typed resources of depth at most D; one parent per node
Assignment	role assignment: principal, role definition, scope	grant: principal, role, resource, validity interval
Inheritance	lower levels inherit role permissions from higher levels	a grant applies at its resource and at every descendant reached through active resources
Multiple assignments	additive	additive
Group membership	transitive	one level; groups do not contain groups
Negative authorization	deny assignments, created by Azure	none
Other conditions	attribute conditions; start and end dates through Privileged Identity Management	validity interval of each grant
Enforcement	evaluated for each request by Azure Resource Manager or integrated data-plane services	evaluated by the application's database as a predicate within each query

is the sequence $r, \mathrm{parent}(r), \mathrm{parent}(\mathrm{parent}(r)), \ldots$ up to the root, which includes r. The active ancestor path $\mathrm{anc}^+(r)$ is the longest prefix of $\mathrm{anc}(r)$ whose resources are all active; it is empty when r is inactive.

Principals are users, groups, service accounts and agents. For a principal p, $\mathrm{resolve}(p)$ contains p and, if p is a user, the active groups of which p is a member; groups do not contain groups. We write $M = |\mathrm{resolve}(p)|$ for the number of principals whose grants count for p. A role is a set of permissions, $\mathrm{perms}(\mathit{role})$, and a permission may be restricted to one resource type. A grant $(p, \mathit{role}, x, t_1, t_2)$ assigns a role to principal p at resource x for the interval $[t_1, t_2]$, either bound of which may be absent; it is active at time t if t lies in the interval.

Definition 1 (Access decision). For a principal p, permission perm, resource r and time t, $\mathrm{allowed}(p, \mathit{perm}, r, t)$ holds if and only if there exist $p' \in \mathrm{resolve}(p)$, a resource $x \in \mathrm{anc}^+(r)$ and a grant $(p', \mathit{role}, x, t_1, t_2)$ active at t with $\mathit{perm} \in \mathrm{perms}(\mathit{role})$, and, if perm is restricted to a resource type, r has that type.

An application table takes part in authorization when each of its rows names a resource in a column ResourceId. In the retail example, each product row names a resource, either its store or a resource of its own below the store. We call such a table protected and write $\mathrm{res}(r)$ for the resource of row r. A row r is authorized for p and perm at t if $\mathrm{allowed}(p, \mathit{perm}, \mathrm{res}(r), t)$ holds.

C. The Access Check in SQL

In the implementation of [9], the access check is an inline table-valued function, `fn_IsResourceAccessible`, which we call the access function. It takes a resource identifier, the principal identifiers of the caller (the user or agent that makes the request) and a permission identifier. A recursive common table expression walks from the resource toward the root for at most D steps and joins each step with the active grants of the given principals whose roles carry the permission. Because the function is inline, the optimizer substitutes its body into the calling query, so the check runs as part of the query's own plan, as if the application had written it by hand. A query keeps only authorized rows with the condition `EXISTS (SELECT 1 FROM fn_IsResourceAccessible(T.ResourceId, ...))`. The function is sound and complete with respect to Definition 1 for hierarchies within the depth bound, and one evaluation costs $O(D \cdot M \cdot G_{\max})$ index lookups, where $G_{\max}$ is the largest number of active grants that one principal holds at one resource [9]. For a page of P rows with cursor pagination on an indexed sort key, [9] gives $O(P \cdot D \cdot M \cdot G_{\max})$ when nearly all rows are authorized, $O(P/\sigma \cdot D \cdot M \cdot G_{\max})$ when a fraction σ of them is, and $O(N \cdot D \cdot M \cdot G_{\max})$ when none is. The method of Section IV uses the function unchanged in its first and last steps.

Notation. N is the number of rows of the paged table, P the page size, D the depth bound of the hierarchy, and $L(r) = |\mathrm{anc}^+(\mathrm{res}(r))| \le D+1$ the number of active ancestors of a row's resource. For a given caller and permission, V is the number of authorized rows, $\sigma = V/N$ the caller's selectivity, G the number of active grants held by the caller's principals whose roles carry the permission, and k the number of minimal granted scopes (Section IV-C). R is the number of rows read by checking rows in sort order (Section III-B), and I the number of projected orderings of a table (Section IV-B). As in [9], an index seek counts as one operation; with B-tree indexes, a seek costs $O(\log N)$ page accesses.

III. SORTED PAGES OF AUTHORIZED ROWS

A. Problem Statement

A list screen asks for the first P rows that a user may see in a chosen order, and then for the P rows after the last one shown. Let T be a protected table with primary key κ. An ordering $K = (c_1, \ldots, c_m, \kappa)$ is a sequence of columns of T that ends with the primary key, so that rows with equal values in the other columns still have a fixed order, and K therefore orders the rows of T totally; we write $r <_K s$, and $K(r)$ for the values of K in row r. A paged query $Q = (T, \varphi, K)$ restricts T by an application-defined filter φ, such as a match on the product name, and orders it by K, ascending or descending. A cursor u is either empty or a tuple of values of K.

Definition 2 (Authorized page). For a caller p, permission perm, time t, cursor u and page size P, the authorized page of

Q is the sequence of the first P rows r of T in K order such that $K(r)$ follows u, $\varphi(r)$ holds, and r is authorized for p and $perm$ at t. If fewer than P rows qualify, the page contains all of them.

Each page is returned with a cursor that records where the page ended in K order, and the next request continues after that position. This is keyset pagination. The database seeks to the cursor's position in the index instead of counting past earlier rows, so the cost of a page does not depend on how many pages came before it [9]. Requesting pages until one is returned without a cursor lists every authorized row of Q in K order.

A solution has to leave the application's tables as they are and keep every authorization decision inside the database. The application owns T, so a method may add only the ResourceId column and an index on it, and anything else lives in separate tables. A statement receives only the caller's principal identifiers and the permission identifier, and any set derived from the caller's grants, such as the nodes where the caller holds grants, is computed inside the statement, because its size is not bounded. Derived structures are updated in the transaction of each write that changes them, and every returned row satisfies Definition 1 when the statement executes. The goal is a cost per page that is bounded independently of N and of the caller's selectivity.

B. Checking Rows in Sort Order

Checking rows in sort order costs one row read for every row up to the P-th authorized row. This is the method of [9], which reads the rows of Q after u through an index on K and evaluates the access function on each until P pass. Let $R(u)$ be the number of rows of Q after u, up to and including the P-th authorized row, or all rows of Q after u if fewer than P are authorized. The page costs $\Theta(R(u))$ row reads and $R(u)$ evaluations of the access function, each $O(D \cdot M \cdot G_{\max})$.

$R(u)$ depends on where the caller's rows fall in K order, not on the cost of a check. If the caller is authorized for a fraction σ of the rows of Q, spread evenly in K order, $R(u)$ is about P/σ. If the caller's V authorized rows are the last V rows of Q, the first page has $R = N - V + P$. Fig. 1 shows both cases in the workload of Section VII, with $R = 184{,}205$ for a caller with 100 rows spread through the order and $R = 980{,}020$ for a caller whose 20,000 rows come last. An index on ResourceId makes each check faster but does not reduce the number of rows checked.

C. Reading the Caller's Rows Through an Ancestor Closure

Reading the caller's rows directly skips the rows the caller may not see, but it must read all of the caller's rows before it can sort them. The ancestor closure of T is a table that pairs every row with every active ancestor of its resource, the set of pairs (x, r) with $r \in T$ and $x \in \mathrm{anc}^{+}(\mathrm{res}(r))$, indexed by x. With it, the 100 products below a store, or the 2,000 below a region, are found with one index lookup on the store or the region. The caller's authorized rows are the rows paired with the nodes where the caller holds a qualifying grant, so they can be read without touching other rows. The closure does not sort them by K, so returning the first P in K order means reading every authorized row after u that satisfies φ and keeping the first P, which costs $\Theta(V)$ row reads for the first page. For the chain manager, the 20 cheapest products of the chain can only be found by reading all 20,000 and sorting them. Position no longer matters, but the number of rows the caller may see does.

Running both methods in rounds with growing budgets and taking whichever finishes first reads $O(\min(R, V) + P)$ rows, which still grows with the table for some callers. For the caller whose 20,000 rows come last, $V = 0.02N$ and $R = N - V + P$, so both terms grow linearly with N when the caller's share of the table is fixed.

Choosing one method in advance would require knowing R and V, and both depend on the caller's grants. A query optimizer chooses a plan from statistics about the table, such as how prices are distributed. These statistics are the same for every caller, so they cannot tell the optimizer whether this caller's rows come early or late in the order, or how many there are. Both dependencies disappear if the rows below each node can be read already sorted by K, which is what Section IV provides.

IV. Ancestor-Partitioned Index Projections

A. Definition

A projection lists every row of a table once under each of its ancestors, sorted by an index key within each ancestor, so that the rows below any node can be read in sorted order with one range scan, a read of consecutive entries of an index. Fig. 2 shows a projection on $(\mathrm{Price}, \mathrm{Id})$ for four products. Products p1 and p2 belong to store S1, products p3 and p4 to store S2, and both stores to region R1. The projection lists p1 and p2 under S1 and p3 and p4 under S2, each pair in price order, and it lists all four products under R1, again in price order. A manager of R1 therefore finds the region's products, already sorted by price, in the consecutive entries under R1. The chain C1, the division D1 and the root hold the same four entries, because they are ancestors of the same products.

Definition 3 (Projection). For a protected table T and an ordering K, the ancestor-partitioned index projection of T on K is the relation

$$\Pi_K(T) = \{\, (x, K(r)) : r \in T,\ x \in \mathrm{anc}^{+}(\mathrm{res}(r)) \,\}.$$

It is stored in a separate table with primary key $(x, c_1, \ldots, c_m, \kappa)$, which keeps the entries under one node together and in K order, and a secondary index on κ, which finds all entries of a row when the row changes. A stored projection is consistent with T and the resource tables if it contains exactly these entries.

A row has an entry under exactly the nodes at which a grant could authorize it under Definition 1, the active ancestors of its resource. Each row therefore has $L(r) \le D + 1$ entries in each projection, and a row whose resource is inactive has none.

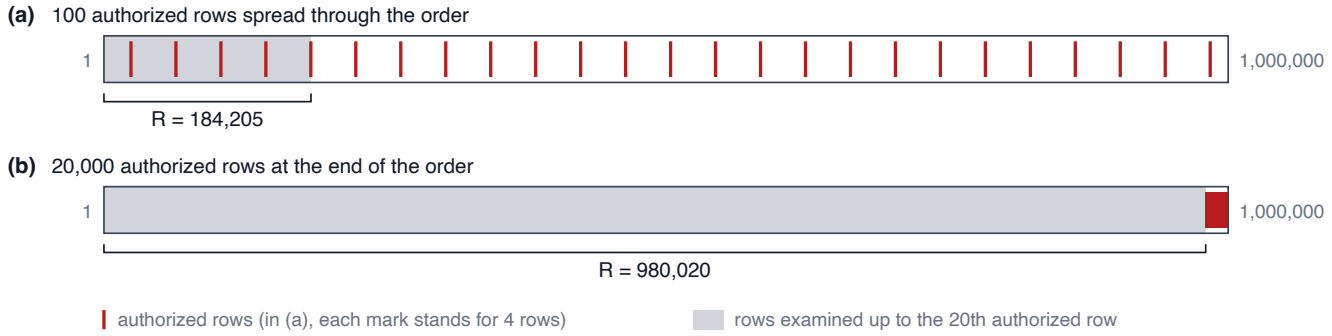
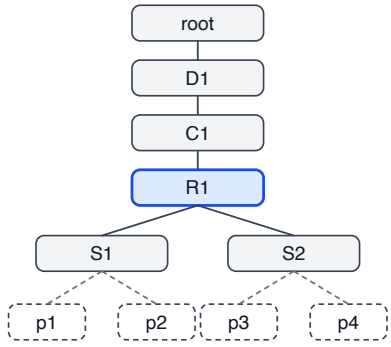


Fig. 1. Rows read by checking rows in price order for a first page of 20, in a table of 1,000,000 products on PostgreSQL 16. (a) A store manager with 100 products spread through the order. (b) A caller whose 20,000 products have the highest prices.

(a) Resource tree



(b) Products

Id	Price	ResourceId
p1	3	S1
p2	9	S1
p3	5	S2
p4	7	S2

(c) Projection on (Price, Id)

Node	Price	Id	
S1	3	p1	
S1	9	p2	
S2	5	p3	
S2	7	p4	
R1	3	p1	range of R1
R1	5	p3	
R1	7	p4	
R1	9	p2	
C1, D1, root: as R1			

Fig. 2. A projection on (Price, Id). (a) The resource tree; dashed boxes are rows of the Products table, whose ResourceId names store S1 or S2. (b) The Products table. (c) The projection lists each product under each active ancestor of its store, sorted by (Price, Id) within each node. The highlighted entries are the range of region R1, and C1, D1 and the root hold the same four entries.

B. Which Orderings Are Projected

Every index of a protected table gets a projection, so an application chooses the orders its lists can use by declaring indexes, as it would without authorization. The primary key defines one projected ordering, and every other index defines the ordering that consists of its columns followed by the primary key. An index that already ends with the primary key defines the same ordering as the index without it, and uniqueness, filters, descending columns and included columns are ignored. Because a B-tree can be read in either direction, one projection serves ascending and descending pages. Indexes that contain ResourceId and indexes whose columns are exactly the columns of a foreign key, such as an index on StoreId that finds a store's products, serve lookups rather than sorting, and they are not projected. An index with a column that cannot be part of a projection key, such as a nullable column, is not projected either.

A query uses a projection only if its ordering, with the primary key appended, matches the projection's columns exactly, so an ordering by name and then city needs an index on (name, city). Without a matching projection, the page is computed by the scan step of the algorithm alone, at a cost of $O(R)$ rows.

C. Minimal Granted Scopes

A page needs only the highest nodes where the caller holds a grant, because a grant below another grant adds no rows. A user with grants on region R1 and on store S1 inside it sees nothing through S1 that R1 does not already give, so the page needs only the entries under R1.

Definition 4 (Minimal granted scopes). For a caller p, permission $perm$ and time t, a resource x is a qualifying scope if it is active and some $p' \in \mathrm{resolve}(p)$ holds a grant at x that is active at t and whose role carries $perm$. $S(p, perm, t)$ is the set of qualifying scopes x such that no resource of $\mathrm{anc}^+(x)$ other than x is a qualifying scope. We write $k = |S(p, perm, t)|$.

A table-valued function, the scope function, computes S inside the statement from the principal identifiers and the permission identifier alone. It applies the same principal, role, validity and activity conditions as the access function and drops every qualifying scope that has a qualifying scope among its active ancestors.

D. Computing a Page

A page takes three steps (Algorithm 1). A short scan in sort order finishes the page for callers who may see most rows, a merge of the caller's ranges finds it for everyone else, and a

Algorithm 1 Computing a page with ancestor-partitioned index projections.

```
ALGORITHM AuthorizedPage(Q, p, perm, u, P)
  // Q = (T, phi, K); u is the cursor
  pr <- resolve(p); q <- lookup(perm); b <- P
  LOOP
    // Scan
    rows <- first b rows of Q after u in K order,
           each with a(r) = AccessFn(res(r), pr, q)
    A <- those rows with a(r) true
    IF |A| >= P: sel <- first P rows of A; EXIT LOOP
    IF |rows| < b: sel <- A; EXIT LOOP
    // Merge
    IF a projection Pi_K exists:
      FOR EACH x IN ScopeFn(pr, q):
        E[x] <- first b entries of Pi_K under x
                after u whose row satisfies phi
      sel <- first P entries of the union of
             all E[x], in K order
      EXIT LOOP
    b <- 4 * b
  // Verification
  page <- the rows of Q with keys in sel and
          AccessFn(res(r), pr, q), in K order
  cursor <- none if the scan or the merge ran
            out of rows, else K(last entry of sel)
  RETURN (page, cursor)
```

final check with the access function admits only authorized rows. The steps run in rounds with a budget b, the number of rows or entries that a step may read in one round, which starts at P.

In Algorithm 1, AccessFn is the access function of Section II-C, ScopeFn is the scope function of Section IV-C, and sel holds the values of K, including the primary key, of the rows selected for the page.

The scan finishes the page whenever authorized rows are common at the start of the order. It reads the next b rows of Q in K order after u and evaluates the access function on each in the same statement. If at least P of them are authorized, the first P form the page; if fewer than b rows remain, the query is exhausted and its remaining authorized rows form the last page. Callers who may see most rows get their page here, as do queries whose filter already restricts T to one of the caller's scopes, such as a list of one store's products.

The merge finds the page when authorized rows are rare or come late. It runs when the scan does not finish the page and a projection exists for K. For each $x \in S$, it reads the first b entries of Π_K under x that follow u and whose row satisfies φ, and it merges these ranges in K order, keeping the first P entries. For a manager of two stores, it reads the first 20 entries under each store and interleaves them by price, as in the merge step of merge sort. Because $b \geq P$, the merge finishes the page in the round in which it runs. Either it returns P entries, or every range is exhausted and it returns all remaining rows below the caller's scopes. The budget grows to $4b$ only for orderings without a projection, for which the scan is repeated.

Verification guarantees that only authorized rows are returned, whatever the projections contain. It loads the selected rows of Q by key, evaluates the access function on each, and returns the rows that pass, in K order. The cursor holds the values of K for the last row selected by the scan or the merge, so rows removed by verification are not selected again by the next page. It is empty when the scan reached the end of Q or every range was exhausted. With a projection, a page issues at most five statements, namely two that resolve the principals and the permission, the scan, the merge and the verification.

Fig. 3 traces the three steps for a store manager in the workload of Section VII.

E. Keeping Projections Current

Every write that changes a row or the hierarchy updates the projections in its own transaction, so a committed write is always reflected and a rolled-back write leaves nothing behind. Inserting or deleting a row, or changing its ResourceId or a projected column, deletes the row's entries through the index on κ and inserts entries for $\mathrm{anc}^+(\mathrm{res}(r))$. Moving a resource to a new parent, or changing whether it is active, changes anc^+ for every row below it. When a region moves to another chain, for example, every product in the region has a new chain and a new division above it. The entries of those rows are recomputed by walking the resource's subtree. Writes that bypass this mechanism, such as bulk loads, require a rebuild that recomputes each projection from T and the resource tables, and the same procedure fills the projection of a new index. Fig. 4 shows the entries written by an insertion and by a move.

V. Correctness and Cost

A. Soundness

The method returns only authorized rows, whatever the projections contain, because every returned row passes the access function.

Theorem 1 (Soundness). For any contents of the projections, every row returned by Algorithm 1 is authorized for the caller and the permission at the time verification executes.

Proof. Rows are returned only by verification, which evaluates the access function on each selected row. By Theorem 1 of [9], the access function returns a result only for rows that satisfy Definition 1. The scan and the merge determine which rows are selected, and no selected row is returned without passing verification. □

B. Completeness

When the projections are consistent, the method returns exactly the page of Definition 2, and its cursor neither skips nor repeats a row. Two lemmas connect the projection to Definition 1.

Lemma 1 (Projection membership). If Π_K is consistent, then for every row r and resource x, Π_K contains an entry for r under x if and only if $x \in \mathrm{anc}^+(\mathrm{res}(r))$, which is the condition that Definition 1 places on the resource of a grant that authorizes r.

Proof. Immediate from Definition 3 and the definition of consistency. □

Lemma 2 shows that every authorized row lies below exactly one minimal granted scope, so the ranges that the

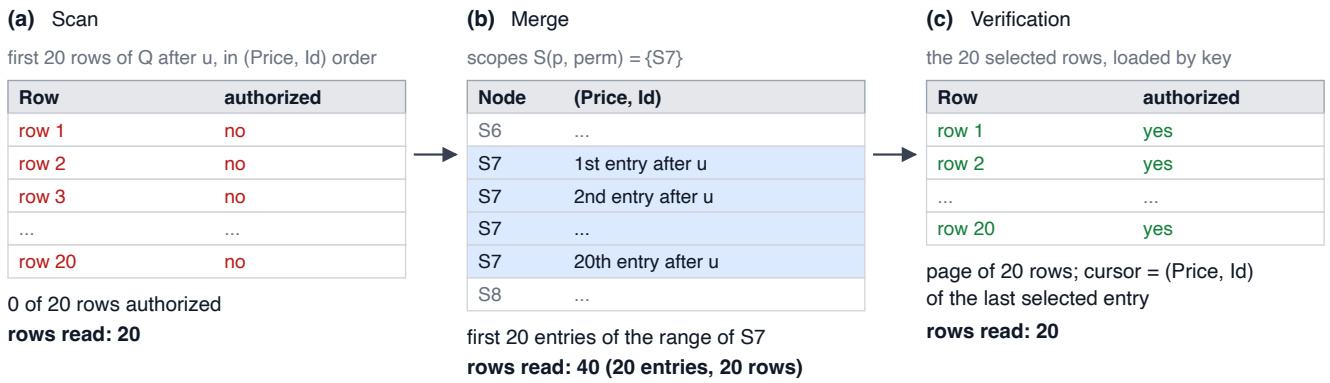


Fig. 3. A first page for a store manager in the workload of Section VII. (a) None of the first 20 rows in price order belongs to the caller's store. (b) The scope function returns the store, S7, and the merge reads the first 20 entries of its range and the corresponding rows. (c) Verification checks the 20 selected rows. The page reads 80 rows on PostgreSQL 16.

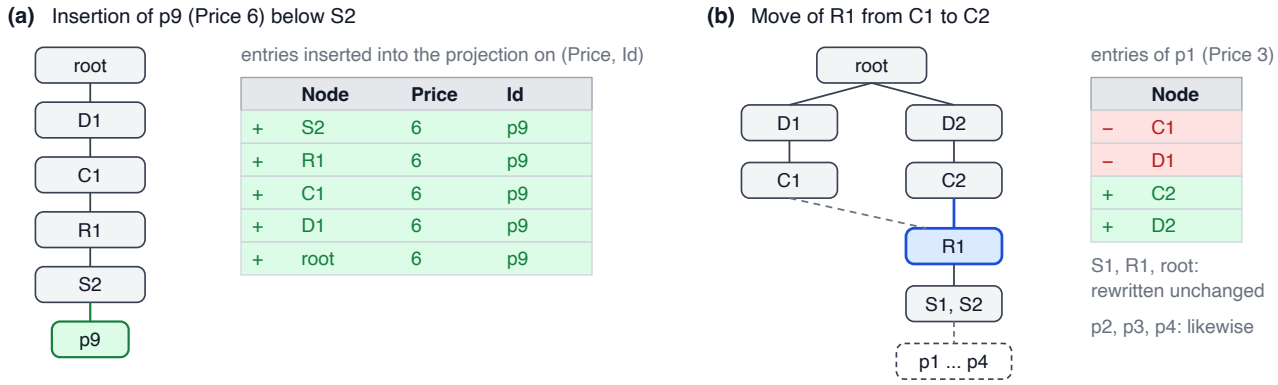


Fig. 4. Maintenance in the example of Fig. 2. (a) Inserting product p9 (price 6) below store S2 adds one entry per active ancestor of S2 to every projection. (b) Moving region R1 from chain C1 to chain C2 rewrites the entries of every row below R1; for p1, the entries under C1 and D1 become entries under C2 and D2. Both changes run in the write's transaction.

merge reads contain all of the caller's rows and none of them twice.

Lemma 2 (Disjoint cover). Let $perm$ be unrestricted or restricted to the type of $\mathrm{res}(r)$. Then r is authorized for p and $perm$ at t if and only if some element of $S(p, perm, t)$ lies in $\mathrm{anc}^{+}(\mathrm{res}(r))$, and at most one element of $S(p, perm, t)$ does.

Proof. If r is authorized, a qualifying scope y lies in $\mathrm{anc}^{+}(\mathrm{res}(r))$. Let y^{*} be the qualifying scope on this path that is closest to the root. Since $\mathrm{anc}^{+}(y^{*})$ is the part of the path from y^{*} upward, it contains no other qualifying scope, so $y^{*} \in S$. Conversely, an element of S on the path is a qualifying scope, so r is authorized. If two elements of S lay on the same path, the higher one would belong to the active ancestor path of the lower one, which contradicts the minimality of the lower one. □

Theorem 2 (Completeness). Suppose Π_K is consistent and $perm$ is unrestricted or restricted to a type shared by the resources of all rows of T. Then Algorithm 1 returns the authorized page of Definition 2, and the cursor it returns continues the enumeration without omitting or repeating an authorized row.

Proof. If the scan finishes the page, it has read the rows of Q after u in K order and evaluated the access function on each, which is sound and complete [9]; its selection consists of the first P authorized rows, or all of them if the query is exhausted. Otherwise the merge runs. By Lemmas 1 and 2, the authorized rows after u that satisfy φ are the disjoint union, over $x \in S$, of the rows with an entry under x after u that satisfy φ. Each range lists the rows of its scope in K order, so each of the first P rows of the union is among the first P rows of its own range, and reading $b \geq P$ entries from every range and merging them yields the first P rows of the union. If the union contains fewer than P rows, every range is exhausted and all of them are returned. By the same lemmas every selected row is authorized, so verification removes none. The selection contains every authorized row up to its last element, which therefore marks the position from which the next page continues. □

If its assumptions fail, the method can return a short page or miss rows, but it still returns only authorized rows. If $perm$ is restricted to a type that some rows below a scope do not have, the merge can select such rows and verification removes them,

so the page may hold fewer than P rows. The selection still contains every authorized row up to its last element, so the enumeration omits none. If a projection is not consistent, for example after a write that bypassed maintenance, an authorized row whose entry is missing does not appear until the next rebuild, and entries for rows that are no longer authorized are removed by verification. Because the steps are separate statements, a grant or membership revoked between the merge and verification also removes rows from the page, which then reflects the authorization state at verification.

C. Page Cost

With a projection, the cost of a page depends on the page size, the depth of the hierarchy and the caller's principals, grants and scopes, but not on the table size or on the caller's share of it.

Theorem 3 (Page cost). For an ordering with a projection and a query without a filter (φ true for every row), a page costs $O(P \cdot D \cdot M \cdot G_{\max})$ operations in the scan, $O(G \cdot D)$ to compute S, $O(k \cdot P \log k)$ in the merge and $O(P \cdot D \cdot M \cdot G_{\max})$ in verification, a total of $O(P \cdot D \cdot M \cdot G_{\max} + G \cdot D + k \cdot P \log k)$, which is independent of N and σ.

Proof. With a projection, Algorithm 1 completes in its first round, with $b = P$. The scan reads at most P rows through the index on K and evaluates the access function on each. The scope function examines the caller's G qualifying grants and, for each granted resource, at most D active ancestors. The merge performs one seek per scope into the primary key of Π_K, reads at most P entries per scope and the corresponding rows, and merges at most $k \cdot P$ entries with a heap of size k. Verification reads at most P rows by key and evaluates the access function on each. None of these terms depends on N or σ. □

For the store manager, $k = 1$ and the merge reads 20 entries, and the whole page reads 80 rows on PostgreSQL (Section VII); for a caller with grants on 100 stores, $k = 100$ and the merge reads up to 2,000 entries. A filter that only a fraction f of a scope's rows satisfy makes the merge read about P/f entries of that range, and at most all of its rows. The scan runs first because it costs at most P rows and finishes the page for callers who may see most rows, without the scope function or a projection.

D. Storage and Write Cost

Storage grows with the number of rows, ancestors and projected orderings, and a write touches only the entries of the rows it changes.

Proposition 1. For a table of N rows with I projected orderings, the projections contain at most $N \cdot (D+1) \cdot I$ entries. Inserting or deleting a row, or changing its ResourceId or a projected column, writes at most $(D+1) \cdot I$ entries. Moving a resource, or changing whether it is active, rewrites the entries of the N_x rows below it, at most $N_x \cdot (D+1) \cdot I$ entries. A rebuild writes every entry once.

Proof. Each row has $L(r) \leq D+1$ entries in each projection, and a write changes only the entries of the rows whose anc^+ or projected values it changes. □

E. Cases Outside the Bound

The bound grows with the number of scopes, so callers with many separate grants are expensive. A caller with 10,000 grants on individual rows has $G = k = 10{,}000$, and the merge performs 10,000 seeks to find 20 rows (Section VII-E). A filter that few of the caller's rows satisfy increases the entries read per scope, as described in Section V-C, and an ordering without a projection is computed by the scan alone, at a cost of $O(R)$ rows.

VI. Implementation

SqlOS [18], an open-source library that provides SHRBAC to applications built on Entity Framework (EF) Core, the object-relational mapper of .NET, implements the method for SQL Server and PostgreSQL. Both engines use the same tables, statement shapes and tests, with engine-specific SQL where the dialects differ.

A. What an Application Declares

An application declares indexes as it normally would and reads pages through one method, and the library creates and maintains the projections.

```
modelBuilder.Entity<Product>(e =>
{
    e.HasIndex(p => p.ResourceId);
    e.HasIndex(p => new { p.Price, p.Id });
    e.HasIndex(p => new { p.Status, p.CreatedAt });
});

var page = await db.Products
    .Where(p => p.Name.Contains(term))
    .OrderBy(p => p.Price)
    .ToPageAsync(principalId, "product.view",
                 pageSize: 20, cursor);
```

ToPageAsync returns the page together with a cursor, and the application passes that cursor back to request the following page. The library adds each projection to the application's EF Core model as a table in its own schema, so the application's migrations or schema tools create it with the application's tables. It derives the projected orderings from the index declarations of every entity type that carries a ResourceId, by the rules of Section IV-B. Projection columns copy the type, length, precision, collation and value conversion of their source columns, so a projection sorts values as the source index does. An index or entity type that cannot be projected, for example because a column is nullable or the entity is mapped to a view, is reported once at startup. A query whose ordering has no projection is served by the scan step, and the library logs a warning that names the index to declare.

B. SQL Statements

Each step of Algorithm 1 is one SQL statement, and no set derived from the caller's grants leaves the database. The scan evaluates the access function in the select list of the application's query, as an EXISTS expression, and limits the query to b rows. The merge joins the scope function, `fn_PageNodes`, to a correlated subquery, which the database evaluates once for each scope and which reads the first b projection entries after the cursor whose row satisfies φ.

On SQL Server this is CROSS APPLY with TOP and on PostgreSQL JOIN LATERAL with LIMIT, so each scope costs one bounded range scan of the projection's primary key. EF Core translates the simpler formulation of a per-scope limit into a join with a window function that numbers every entry of the projection, which reads the whole projection before any limit applies, so the library builds the correlated subquery itself. Verification passes the selected keys as one parameter, through OPENJSON on SQL Server and an array on PostgreSQL. Two statements resolve the caller's principals and the permission at the start of every page, so each page sees the grants and group memberships committed before it. The ancestor walk returns no rows for a resource whose chain exceeds the configured depth or contains a cycle, so the rows of such a resource are neither authorized nor projected.

The scope function is written differently on the two engines because the same formulation was slow on PostgreSQL. SQL Server uses a correlated NOT EXISTS. On PostgreSQL, that formulation rescanned the materialized set of granted resources once for each granted resource and took 6.5 s for a caller with 10,000 grants, whereas a set difference (EXCEPT) took 0.3 s. The PostgreSQL function also declares a fixed row estimate and a SET clause that disables just-in-time compilation, the step in which PostgreSQL compiles an expensive query to machine code before running it, which costs more than it saves on short queries. A function with a SET clause is not inlined into the calling query; when it was inlined, the estimated cost of page queries exceeded PostgreSQL's compilation threshold, and compilation added about 500 ms to pages whose execution took about 20 ms.

C. Maintenance in the Save

Projections are maintained inside the application's save, so a write and its projection entries commit or roll back together. Before a save, the library's database context checks whether the pending changes include a resource or a row of a protected table, and if they do not, the save proceeds unchanged. Otherwise the context opens a transaction or, if the application already has one, a savepoint within it, so that the library can undo its own work without ending the application's transaction. It then performs the save, collects the rows and resources that the save inserted, deleted or modified, and issues one command that updates every affected projection. Collecting changes after the save captures values set by save interceptors, cascade deletions and keys generated by the database. A failure in either step rolls back both.

Two implementation issues emerged during testing with one million rows on PostgreSQL. The estimated cost of the subtree statements, a recursive walk combined with the inlined ancestor function, exceeded the threshold for just-in-time compilation, and inserting a single row took 165 ms instead of 1.6 ms. The maintenance command therefore disables just-in-time compilation for its own statements and restores the transaction's setting afterwards. Walking subtrees from every changed resource, including resources added in the same save, led the optimizer to estimate a save of 1,000 new resources at millions of rows and to hash the entire projection. Because a resource's parent is a foreign key, a resource added or deleted in a save has no descendants outside that save, so the implementation walks only resources whose parent or activity changed. The maintenance command for that save fell from 2.6–4.9 s to 66–111 ms.

Writes that bypass the save are repaired by a rebuild, which deletes and rewrites each projection in its own transaction, under an application lock and without a command timeout. At startup, the library fills every projection that is empty while its source table has rows, which is the state after an index is added.

VII. Evaluation

A. Research Questions

- **RQ1.** How does the cost of a page depend on the caller's selectivity and on the position of the caller's rows in the sort order, for checking rows in order and for the proposed method?
- **RQ2.** What does the method add for callers who may see most rows and for queries already restricted to one of the caller's scopes?
- **RQ3.** How does the cost of the method depend on the number of minimal granted scopes?
- **RQ4.** How do sorted projection ranges compare with reading the caller's rows through an unordered ancestor closure?
- **RQ5.** What do the projections cost in storage and in write latency?

B. Experimental Setup

Workload. The workload is a retail hierarchy with a root, 5 divisions, 50 chains, 500 regions, 10,000 stores and 1,000,000 products, with 10 chains per division, 10 regions per chain, 20 stores per region and 100 products per store. Each product is its own resource below its store, so every product has six active ancestors including itself. Prices are uniform over the products, except that the 20,000 products of one chain carry the highest prices. The Products table has indexes on (Price, Id) and (StoreId, Price, Id) and a unique index on ResourceId. Every query orders by (Price, Id), uses a page size of 20 and requests the first page.

Callers. Seven callers span the cases of Section III, from an administrator who may see every product to a caller whose products all sort last (Table II). Each holds grants on the resources named in the table, with a role that carries the permission product.view. The query of the scoped store manager also filters on the caller's StoreId, as a list of one store's products would. For RQ3, five further callers hold grants on 1, 10, 45, 100 and 500 stores whose products are spread through the price order.

Methods. We compare the proposed method with the two methods of Section III.

- **A, checking rows in order** [9]: the application's query with EXISTS over the access function, ordered by (Price, Id) and limited to 20 rows, executed as one statement. As

TABLE II
CALLERS. V IS THE NUMBER OF PRODUCTS THE CALLER MAY SEE, AND R THE NUMBER OF ROWS IN PRICE ORDER UP TO AND INCLUDING THE 20TH OF THEM, MEASURED ON POSTGRESQL.

Caller	Grants	V	Share	R
Administrator	one, on the root	1,000,000	100%	20
Region manager	one, on a region	2,000	0.2%	9,306
Store manager	one, on a store	100	0.01%	184,205
100 stores	one on each of 100 stores	10,000	1%	2,233
10,000 single products	one on each of 10,000 products	10,000	1%	1,452
Far end	one, on the chain with the highest prices	20,000	2%	980,020
Scoped store manager	one, on a store; query filtered by StoreId	100	0.01%	20

in [9], the caller's principals are resolved before timing starts.

- **B, projections**: Algorithm 1, including the two statements that resolve the caller's principals and the permission.
- **C, closure enumeration**: the combination of A with closure enumeration described in Section III-C. C works in rounds with budgets $b = 20, 80, 320, \ldots$, and in each round it checks the next b rows in order, as A does. Once the caller's minimal scopes hold at most b entries in a key-only projection, which is the ancestor closure of Section III-C, C instead reads those rows by key, sorts them by (Price, Id) and returns the first 20 after the cursor. It takes the page from whichever path finishes first, passes the caller's row keys from the closure statement to the fetch statement as a parameter, and reads $O(\min(R, V) + P)$ rows.

Platform. All measurements ran on one Apple Silicon laptop with Docker Desktop. PostgreSQL 16 ran natively with its default settings, including just-in-time compilation. SQL Server 2022 ran as the x64 image under CPU emulation, which adds tens of milliseconds to each statement; its row counts are exact, but its times are not representative of native execution and are compared only within the engine.

Metrics. The main metric is rows read, which is exact on both engines and does not depend on the speed of the machine. It counts the rows produced by the plan operators over Products and the projection tables, summed over all statements of a page and taken from actual execution plans (SET STATISTICS XML on SQL Server; EXPLAIN (ANALYZE, FORMAT JSON) on PostgreSQL, including rows removed by filters). On SQL Server, the plans of A read each examined row twice, once from the price index and once through a key lookup for ResourceId. Time is the median of up to 15 runs within a 60-second budget after a warm-up run; a run longer than one minute was measured once.

Artifacts. The implementation, benchmark harness and data generator are part of SqlOS [18]. The comparison of A and B on both engines (Tables III and IV) was measured with version 10.0.0 [19], in which the price projection was declared explicitly and no other ordering was projected. Version 11.0.0 derives the same projection from the price index and issues the same page statements. A repetition on PostgreSQL with version 11.0.0 reproduced every row count of Table IV, with B times of 14.0, 18.3, 18.6, 24.9, 457.5, 16.4 and 12.4 ms in the order of Table II [20]. Method C and the sweep over granted stores (Tables V and VI) were measured with an experimental build of the same harness on the same machine and data [21], with A and C in one run and B in a second. In these runs, A and B read the same numbers of rows as in Tables III and IV. The row counts of C differed between runs because its plans depend on optimizer statistics; the region manager read 13,263 rows in one run and 12,544 in another. Storage and write costs (Tables VII and VIII) were measured with version 11.0.0 [20].

C. RQ1: Selectivity and Position

Under A, the rows read for a page range from 20 to 980,020 depending on where the caller's rows fall. Under B, no caller with one scope reads more than 80 rows on PostgreSQL or 100 on SQL Server (Tables III and IV, Fig. 5). For the store manager, the rows read fall from 184,205 to 80 on PostgreSQL and from 368,410 to 100 on SQL Server, and the time falls from 8,420 ms to 22.6 ms and from 179,558 ms to 89.0 ms. The region manager reads 80 rows instead of 9,306. The far-end caller, whose products all sort last, reads 980,020 rows under A and 80 under B on PostgreSQL, and 1,960,040 and 100 on SQL Server. Its time falls from 44,208 ms to 16.4 ms and from 942,593 ms to 90.3 ms. These callers' 80 rows are 20 in the scan, 40 in the merge (20 projection entries and the 20 rows on which φ is evaluated) and 20 in verification; SQL Server reads 100 because its scan reads each row twice. Position has no effect under B, because the range of the far-end caller's chain begins with its cheapest product.

D. RQ2: Dense and Scoped Callers

For callers who may see most rows, B adds 7 to 9 ms on PostgreSQL and 21 to 29 ms on SQL Server. The administrator's page takes 14.0 ms instead of 5.6 ms on PostgreSQL and 52.3 ms instead of 23.7 ms on SQL Server, and reads 40 rows instead of 20 on PostgreSQL. The scoped store manager's page takes 12.8 ms instead of 5.4 ms and 39.4 ms instead of 18.4 ms. In both cases the scan finishes the page and B reads no projection. A executes one statement and B four, which resolve the caller's principals and the permission, scan 20 rows and verify them; B's times include the two resolution statements, which A performs before timing starts.

TABLE III
SQL SERVER 2022, 1,000,000 PRODUCTS, FIRST PAGE OF 20 ROWS ORDERED BY PRICE.

Caller	A: time (ms)	A: rows read	B: time (ms)	B: statements	B: rows read
Administrator	23.7	40	52.3	4	60
Region manager	9,032	18,612	88.2	5	100
Store manager	179,558	368,410	89.0	5	100
100 stores	2,227	4,466	93.7	5	4,060
10,000 single products	1,420	2,904	762.7	5	20,060
Far end	942,593	1,960,040	90.3	5	100
Scoped store manager	18.4	40	39.4	4	60

TABLE IV
POSTGRESQL 16, 1,000,000 PRODUCTS, FIRST PAGE OF 20 ROWS ORDERED BY PRICE.

Caller	A: time (ms)	A: rows read	B: time (ms)	B: statements	B: rows read
Administrator	5.6	20	14.0	4	40
Region manager	416.8	9,306	18.6	5	80
Store manager	8,420	184,205	22.6	5	80
100 stores	107.6	2,233	21.8	5	4,040
10,000 single products	70.8	1,452	390.2	5	20,040
Far end	44,208	980,020	16.4	5	80
Scoped store manager	5.4	20	12.8	4	40

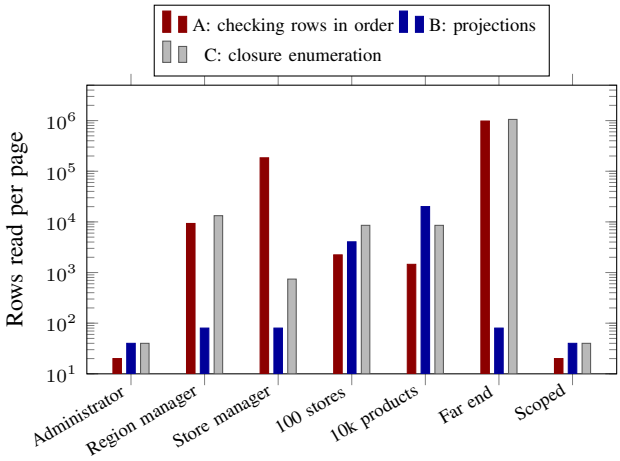


Fig. 5. Rows read for a first page on PostgreSQL 16 by checking rows in order (A), projections (B) and closure enumeration (C); logarithmic scale.

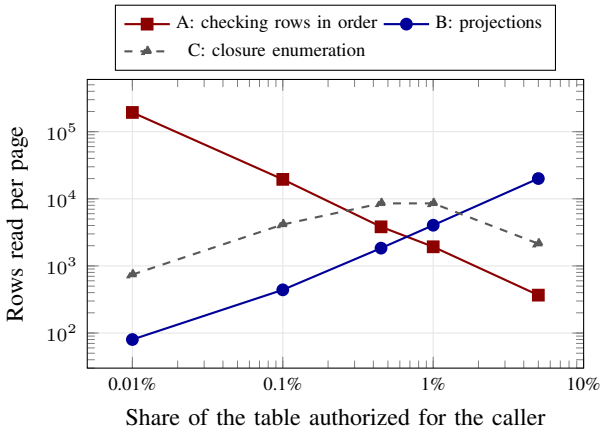


Fig. 6. Rows read for a first page on PostgreSQL 16 against the caller's share of the table, for callers with grants on 1 to 500 stores; logarithmic scales.

E. RQ3: Number of Scopes

The cost of B grows with the number of scopes, at about 40 rows read per scope. The caller with grants on 100 stores reads 4,040 rows under B, 40 for each scope and 40 in the scan and verification, and takes 21.8 ms against 107.6 ms under A on PostgreSQL. The caller with 10,000 grants on single products reads 20,040 rows under B on PostgreSQL, because each of its 10,000 scopes holds one product and the merge reads each one. A reads 1,452 rows for this caller, whose products are spread evenly through the price order, and takes 70.8 ms against 390.2 ms for B. On SQL Server the times are reversed, at 1,420 ms for A and 762.7 ms for B, although B reads 20,060 rows against 2,904.

The sweep over 1 to 500 granted stores shows the trade-off between the two methods (Table VI, Fig. 6). The rows read by A fall from 193,501 to 367 as the caller's share grows, roughly in proportion to P/σ. The rows read by B rise as $40 \cdot k + 40$ for k granted stores, from 80 to 20,040, and its time from 15.5 ms to 38.9 ms. B reads more rows than A from 100 stores on, and takes longer only at 500 stores, where A takes 24.0 ms and B 38.9 ms. A row read costs more under A, which applies the access function to every row it reads, than in B's merge, which reads projection entries and rows without it. A caller with one grant on a higher node, such as the region manager or the far-end caller, reads 80 rows under B whatever its share.

F. RQ4: Projections and Closure Enumeration

Closure enumeration removes the effect of position but still reads all of the caller's rows, and its cost depends on the plan the optimizer picks (Table V). On PostgreSQL, C reads 2.0 to 7.4 times $\min(R, V)$ rows for six of the seven callers, in 4 to 16 statements per page. For the far-end caller, C's fetch statement requested the caller's 20,000 rows by key. PostgreSQL's cost estimates for a parallel sequential scan of

TABLE V

CLOSURE ENUMERATION (C) ON BOTH ENGINES, 1,000,000 PRODUCTS, FIRST PAGE OF 20 ROWS ORDERED BY PRICE. THE LAST COLUMN REPEATS THE ROWS READ BY B FROM TABLES III AND IV.

Caller	$\min(R, V)$	C PostgreSQL: time (ms)	rows read	C SQL Server: time (ms)	rows read	B rows read (PostgreSQL / SQL Server)
Administrator	20	13.7	40	50.9	60	40 / 60
Region manager	2,000	951.3	13,263	2,644.3	19,364	80 / 100
Store manager	100	59.3	742	245.1	1,162	80 / 100
100 stores	2,233	875.2	8,549	2,701.8	15,364	4,040 / 4,060
10,000 single products	1,452	2,305.1	8,544	35,526.3	15,364	20,040 / 20,060
Far end	20,000	3,044.9	1,054,147	12,116.5	101,445	80 / 100
Scoped store manager	20	13.8	40	53.2	60	40 / 60

TABLE VI

SWEEP OVER THE NUMBER OF GRANTED STORES ON POSTGRESQL 16, FIRST PAGE OF 20 ROWS ORDERED BY PRICE. A AND C WERE MEASURED IN ONE RUN AND B IN A SECOND (SECTION VII-B).

Granted stores	V	Share	$\min(R, V)$	A: rows read	A: time (ms)	B: rows read	B: time (ms)	C: rows read	C: time (ms)
1	100	0.01%	100	193,501	11,707.8	80	15.5	742	59.3
10	1,000	0.1%	1,000	19,481	1,295.4	440	16.3	4,142	168.6
45	4,500	0.45%	3,825	3,825	204.3	1,840	17.8	8,549	883.3
100	10,000	1%	1,928	1,928	105.5	4,040	20.2	8,549	899.6
500	50,000	5%	367	367	24.0	20,040	38.9	2,142	200.0

Products and for 20,000 primary-key lookups differed by 7%, the optimizer chose the scan, and the page read 1,054,147 rows. With parallel query disabled, the same page read 74,145 rows in 2,603.1 ms. On SQL Server, C read 101,445 rows for this caller. For the caller with 10,000 single grants, C recomputes the minimal scopes in each round; on SQL Server, the row limit of the closure statement propagated into the minimality computation, and one round took 25 s of the 35,526.3 ms.

Where C reads 742 to 1,054,147 rows for the region manager, the store manager and the far-end caller, B reads 80 for each. C must read all of the caller's rows before it can sort them, whereas B reads the first 20 entries of a sorted range. For the caller with 10,000 single grants, C reads fewer rows than B, 8,544 against 20,040, but takes 2,305.1 ms against 390.2 ms. In the sweep (Table VI), C reads fewer rows than A for the callers with 1 and 10 stores and more for those with 45, 100 and 500, and more than B for every caller except the one with 500 stores. Sorting each scope's entries by the index key, as the projections do, removes the dependence on both the position and the number of the caller's rows and leaves a dependence on the number of scopes.

G. RQ5: Storage and Write Cost

Each projection of the 1,000,000-product table holds 6,000,000 entries and occupies 550 MB to 866 MB, and three projections make a single insertion 4.3 ms slower (Tables VII and VIII). Version 11.0.0 projects the primary key and both indexes. Writing all 18,000,000 entries with the rebuild took 175 s on PostgreSQL; with version 10.0.0, building the price projection alone took 55 s on PostgreSQL and 200 s on SQL Server.

Inserting one product with its resource writes six entries into each projection and takes 5.7 ms against 1.4 ms for an unprotected table. Inserting 1,000 products in one save takes 635.6 ms against 211.8 ms. Moving a region of 2,000 products to another chain rewrites 12,000 entries in each projection and takes 501.0 ms against 5.2 ms. Both columns were measured with the two maintenance changes of Section VI-C.

TABLE VII

PROJECTIONS OF THE PRODUCTS TABLE ON POSTGRESQL 16, 1,000,000 PRODUCTS WITH SIX ACTIVE ANCESTORS EACH.

Ordering	Entries	Size (MB)
(Price, Id)	6,000,000	706
(StoreId, Price, Id)	6,000,000	866
(Id)	6,000,000	550

TABLE VIII

WRITE LATENCY ON POSTGRESQL 16 FOR AN UNPROTECTED PRODUCTS TABLE AND FOR THE SAME TABLE WITH THREE PROJECTIONS (MEDIAN; 25 SAVES FOR THE SINGLE INSERTION).

Operation	Unprotected (ms)	Three projections (ms)
Insert one product	1.4	5.7
Insert 1,000 products in one save	211.8	635.6
Move a region of 2,000 products	5.2	501.0

H. Threats to Validity

The main threats are the single machine, the emulated SQL Server and the synthetic data. All measurements ran on one laptop, and times are medians of a small number of runs on a shared machine; the conclusions rest on the row counts, which are exact. The data has uniform prices, equal fan-out at every level and one resource per product, which gives every product the longest ancestor path of the tree. Skew in the price distribution changes R but not the $k \cdot P$ term of B. All measurements request the first page; deep cursors were not measured, although the analysis of Section V does not depend on the cursor. On PostgreSQL, statements were planned at every execution because the client did not prepare them. The

measurements combine runs of two builds whose statements for A and B are identical, as the identical row counts confirm, and method C is not part of a released version. Integration tests on both engines compare every page of B with row-by-row point checks [18].

VIII. Related Work

A. Authorization Services

Authorization services are built for single checks, and those that list objects return unsorted sets. Zanzibar [8] serves point checks at large scale and describes search as checking candidates, noting that search results "often require tens to hundreds of checks". Clients that need other access paths maintain secondary indexes from its change stream, and its Leopard index precomputes flattened group membership. Services in the Zanzibar family list objects by expanding the relationship graph backwards from the user. OpenFGA's ListObjects returns an unordered set bounded by a result limit and a deadline [10], and its maintainers state that it cannot be paginated because "the graph evaluation is done concurrently (and thus is not ordered)" [22]. Auth0's documentation combines database queries, batch checks and list calls, and notes that a partial list "is not enough, because you won't be able to sort using it" [23].

Services that support sorting copy permissions into the application's database. AuthZed Materialize streams computed permissions into that database so that applications can "sort, filter, and paginate over many thousands of authorized objects natively" [24], and Auth0 FGA documents a precomputed permissions index [23]. These copies hold permission facts and are updated asynchronously from a change stream. Projections hold structure, which rows lie below which node, so they change with rows and resources but not with grants, and they are updated in the writing transaction. Partitioning data by the scopes that a user can read, as projections do, also appears in search systems. Dropbox's search engine determines at query time the namespaces a user may read and retrieves documents within them [25], and Slack keeps a separate indexed copy of a message for each organization that shares a channel [26].

B. Row-Level Security and Query Filters

Row-level security and policy compilers filter rows with predicates. When the authorized set is not a condition on an indexed column, as with grants at arbitrary levels of a hierarchy, their cost follows the rows a query reads, as in Section III-B. Query modification adds authorization predicates to queries [27], and row-level security in PostgreSQL and SQL Server builds it into the engine. PostgreSQL evaluates a policy expression for each row "prior to any conditions or functions coming from the user's query" [28]. Supabase's guidance states that the cost scales with the rows a query scans and recommends indexing the columns that policies filter on and evaluating functions once per statement [29]. Microsoft's guidance for SQL Server row-level security is to avoid recursion in predicate functions [30], and practitioners recommend flattening hierarchies before using them in predicates [31]. Policy engines compile policies into query conditions. Oso generates SQL filters [32], Cerbos returns a condition tree for translation into a query [33], and Open Policy Agent derives filters by partial evaluation [34].

C. Hierarchy Encodings and Merging

Hierarchy encodings make every subtree contiguous in one order, the order of the hierarchy, whereas projections make it contiguous in the order of every index. Closure tables store one row per ancestor–descendant pair, which makes subtree queries simple and makes moves rewrite the pairs of the moved subtree [35]. The hierarchyid type of SQL Server orders nodes depth-first, so a depth-first index places every subtree contiguously [36]; Spanner's interleaved tables store child rows under their parent's key [37]; and nested intervals encode containment in numeric intervals [38]. A projection is a closure table extended with the key of an index and kept once for every projected ordering. The merge step of Algorithm 1 is a k-way merge of sorted ranges. The threshold algorithm of Fagin et al. [39] addresses the more general problem of combining scores from several sorted lists; Algorithm 1 needs only a merge because each authorized row lies in exactly one range.

IX. Limitations and Future Work

The method trades storage and write time for read time. With three projected orderings and six active ancestors, each product has eighteen projection entries, which every insertion writes. Indexes created for filtering, such as (StoreId, Price, Id), are projected as well; that projection occupies 866 MB in Table VII, although no measured query sorts by it, and the implementation has no way to exclude an index. Moving a resource rewrites the entries of every row below it.

Some orderings and grant patterns fall outside the bound. Sort columns must be non-nullable, because they form part of the projection's primary key, and an ordering on a nullable column is served by the scan alone. A caller with many separate grants pays one range read per scope (Section V-E). A permission restricted to a resource type that some rows below a caller's scopes do not have can produce pages with fewer than P rows (Section V-B). In a hierarchy of entity types mapped to one table, only the indexes of the root type are projected.

Projections are only as current as the writes that maintain them. Writes that bypass the library's database context, such as bulk loads and raw SQL statements, leave projections inconsistent until a rebuild. Under snapshot isolation, a save that moves or deactivates a resource and a concurrent save that changes rows below it can leave those rows' entries on the old path. Under locking isolation, either save can fail with a deadlock or a duplicate key. Rows with stale entries can be missing from pages until a rebuild, and verification prevents stale entries from exposing rows; the implementation does not take a lock that every save would contend on.

Several extensions follow from these limitations. Maintaining projections inside the database engine, through statement-level triggers or an extension, would keep them consistent for

every writer; Definition 3 and the rebuild procedure specify the required result. A SQL function that returns an authorized page would make the method available to clients that do not use the library. Encoding nulls as ordered key values would allow nullable sort columns. Omitting the row lookups of the merge when Q has no filter would halve the rows that step reads. Measurements on native SQL Server hardware, larger tables, skewed distributions and deep cursors would complement Section VII.

X. Conclusion

With ancestor-partitioned index projections, the cost of a sorted page of authorized rows no longer depends on the size of the table or on where the caller's rows fall in the order. Checking rows in sort order reads every row up to the P-th one the caller may see, and reading the caller's rows through an ancestor closure reads all of them. Projections keep the rows below every node sorted by every index, so a page reads at most P entries per granted scope, in $O(P \cdot D \cdot M \cdot G_{\max} + G \cdot D + k \cdot P \log k)$ operations, and returns only rows that pass the original access check. With 1,000,000 rows on two engines, callers with one scope read at most 100 rows per page wherever their rows fall. The price is one entry per row, ancestor and projected ordering, written with every change.

AI Disclosure

In accordance with IEEE policy, the author discloses that AI tools (Claude, Anthropic) were used to assist with manuscript preparation and editing and with the implementation and benchmarking work described. The technical decisions and the experimental design are the work of the author.

References

[1] Microsoft, "What is Azure role-based access control (Azure RBAC)?," Microsoft Learn. https://learn.microsoft.com/en-us/azure/role-based-access-control/overview (accessed Oct. 2026).

[2] Microsoft, "Understand Azure role assignments," Microsoft Learn. https://learn.microsoft.com/en-us/azure/role-based-access-control/role-assignments (accessed Oct. 2026).

[3] Microsoft, "Understand scope for Azure RBAC," Microsoft Learn. https://learn.microsoft.com/en-us/azure/role-based-access-control/scope-overview (accessed Oct. 2026).

[4] R. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, "Role-Based Access Control Models," *IEEE Computer*, vol. 29, no. 2, pp. 38–47, 1996.

[5] D. F. Ferraiolo, R. Sandhu, S. Gavrila, D. R. Kuhn, and R. Chandramouli, "Proposed NIST Standard for Role-Based Access Control," *ACM Trans. Information and System Security*, vol. 4, no. 3, pp. 224–274, 2001.

[6] WorkOS, "Fine-Grained Authorization (FGA)," WorkOS Documentation. https://workos.com/docs/fga/index (accessed Oct. 2026).

[7] OpenFGA, "Parent-Child Objects," OpenFGA Documentation. https://openfga.dev/docs/modeling/parent-child (accessed Oct. 2026).

[8] R. Pang, R. Caceres, M. Burrows, Z. Chen, P. Dave, N. Germer, A. Golynski, K. Graney, N. Kang, L. Kissner, J. L. Korn, A. Parmar, C. D. Richards, and M. Wang, "Zanzibar: Google's Consistent, Global Authorization System," in *Proc. USENIX ATC*, pp. 33–46, 2019.

[9] R. Slaney, "Database-Native Authorization for Human and Autonomous Principals: The SHRBAC Model," in *Proc. IEEE COMPSAC*, 2026.

[10] OpenFGA, "Configuration," OpenFGA Documentation (listObjectsDeadline, listObjectsMaxResults). https://openfga.dev/docs/getting-started/setup-openfga/configuration (accessed Oct. 2026).

[11] AuthZed, "Protecting a list endpoint," SpiceDB Documentation. https://authzed.com/docs/spicedb/modeling/protecting-a-list-endpoint (accessed Oct. 2026).

[12] Microsoft, "Organize your resources with management groups," Microsoft Learn. https://learn.microsoft.com/en-us/azure/governance/management-groups/overview (accessed Oct. 2026).

[13] Microsoft, "Understand Azure role definitions," Microsoft Learn. https://learn.microsoft.com/en-us/azure/role-based-access-control/role-definitions (accessed Oct. 2026).

[14] Microsoft, "List Azure deny assignments," Microsoft Learn. https://learn.microsoft.com/en-us/azure/role-based-access-control/deny-assignments (accessed Oct. 2026).

[15] Microsoft, "What is Privileged Identity Management?," Microsoft Learn. https://learn.microsoft.com/en-us/entra/id-governance/privileged-identity-management/pim-configure (accessed Oct. 2026).

[16] Z. Zhang, X. Zhang, and R. Sandhu, "ROBAC: Scalable Role and Organization Based Access Control Models," in *Proc. CollaborateCom*, 2006.

[17] N. Solanki et al., "Resource and Role Hierarchy Based Access Control for Resourceful Systems," in *Proc. IEEE COMPSAC*, pp. 396–401, 2018.

[18] R. Slaney, "SqlOS," open-source library (MIT license), version 11.0.0, Oct. 2026. https://github.com/ross-slaney/sqlos

[19] R. Slaney, benchmark results for SqlOS 10.0.0 on SQL Server 2022 and PostgreSQL 16, ross-slaney/sqlos pull request 516, Oct. 2026. https://github.com/ross-slaney/sqlos/pull/516

[20] R. Slaney, benchmark results for SqlOS 11.0.0 on PostgreSQL 16, ross-slaney/sqlos pull request 524, Oct. 2026. https://github.com/ross-slaney/sqlos/pull/524

[21] R. Slaney, benchmark results for closure enumeration and the selectivity sweep, ross-slaney/sqlos pull request 522, Oct. 2026. https://github.com/ross-slaney/sqlos/pull/522

[22] OpenFGA maintainers, reply on ListObjects pagination, OpenFGA GitHub discussion 105, Jan. 2023. https://github.com/orgs/openfga/discussions/105

[23] Auth0, "Search with permissions," Auth0 FGA Documentation. https://docs.fga.dev/integration/advanced/search-with-permissions (accessed Oct. 2026).

[24] AuthZed, "AuthZed Materialize," AuthZed Documentation. https://authzed.com/docs/authzed/concepts/authzed-materialize (accessed Oct. 2026).

[25] Dropbox, "Architecture of Nautilus, the new Dropbox search engine," Dropbox Tech Blog, Sept. 2018. https://dropbox.tech/machine-learning/architecture-of-nautilus-the-new-dropbox-search-engine

[26] Slack, "Slack search powers visibility and compliance across your organisation," Slack Blog. https://slack.com/blog/developers/enterprise-ai-search-with-slack-search

[27] M. Stonebraker and E. Wong, "Access Control in a Relational Data Base Management System by Query Modification," in *Proc. ACM Annual Conf.*, pp. 180–187, 1974.

[28] PostgreSQL Global Development Group, "Row Security Policies," PostgreSQL Documentation. https://www.postgresql.org/docs/current/ddl-rowsecurity.html

[29] Supabase, "Row Level Security performance," Supabase Documentation. https://supabase.com/docs/guides/database/postgres/row-level-security-performance

[30] Microsoft, "Row-Level Security," SQL Server Documentation. https://learn.microsoft.com/en-us/sql/relational-databases/security/row-level-security

[31] B. Johnston, "SQL Server RLS performance and troubleshooting," Redgate Simple Talk, Sept. 2023. https://www.red-gate.com/simple-talk/databases/sql-server/sql-server-rls-performance-and-troubleshooting/

[32] Oso, "Local Authorization," Oso Documentation. https://www.osohq.com/docs/develop/facts/local-authorization

[33] Cerbos, "Filtering resources by permission," Cerbos Documentation. https://docs.cerbos.dev/cerbos/latest/recipes/filtering-resources

[34] Open Policy Agent, "Data Filtering," OPA Documentation. https://www.openpolicyagent.org/docs/filtering/

[35] B. Karwin, "Moving Subtrees in Closure Table Hierarchies," Percona Blog, Feb. 2011. https://www.percona.com/blog/moving-subtrees-in-closure-table/

[36] Microsoft, "Hierarchical Data (SQL Server)," SQL Server Documentation. https://learn.microsoft.com/en-us/sql/relational-databases/hierarchical-data-sql-server

[37] Google Cloud, "Schemas overview," Spanner Documentation. https://docs.cloud.google.com/spanner/docs/schema-and-data-model

[38] V. Tropashko, "Nested Intervals Tree Encoding in SQL," arXiv:cs/0402051, 2004.

[39] R. Fagin, A. Lotem, and M. Naor, "Optimal Aggregation Algorithms for Middleware," *J. Computer and System Sciences*, vol. 66, no. 4, pp. 614–656, 2003.